

EE69210: Machine Learning for Signal Processing Laboratory
Department of Electrical Engineering, Indian Institute of Technology, Kharagpur

Neural Networks

Anirvan Krishna | Roll no. 21EE38002

Keywords: Fully connected networks, Dropouts, Batch Normalization, Classification

Grading Rubric

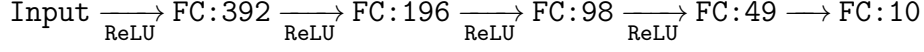
	Tick the best applicable per row			Points
	Below Ex- pectations	Lacking in Some	Meets all Ex- pectations	
Completeness of the report				
Organization of the report (5 pts)				
Quality of figures (5 pts)				
Dataset Loading and explanation (10 pts)				
Model training, inference, and ex- pla- nation for the base model with discus- sions. (25 pts)				
Model training, inference, and expla- nation for the model with dropouts with discussions. (25 pts)				
Model training, inference, and ex- pla- nation for the model with batch nor- malization with discussions. (25 pts)				
TOTAL (100 pts)				

1. Training Details

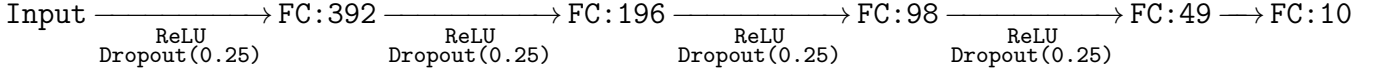
1.1 Model Architecture

A fully connected network defined as $\text{net}(\cdot)$ is used in this study. The $\text{net}(\cdot)$ maps $\mathbb{R}^{784 \times 1} \rightarrow \mathbb{R}^{10 \times 1}$ and is defined as follows:

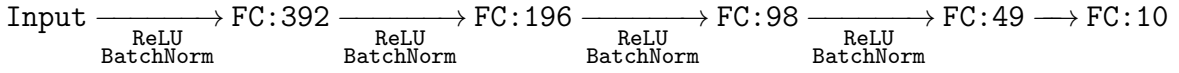
Model-1:



Model-2 (with Dropout):



Model-3 (with BatchNorm):



1.2 Classification Loss Function

The classification task in this study uses the cross-entropy loss function, which is defined for a single sample as:

$$\mathcal{L}_{\text{CE}} = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

where C is the number of classes, y_i is the ground truth label (one-hot encoded), and $\hat{y}_i$ is the predicted probability for class i (usually obtained after applying the softmax function). The loss encourages the predicted distribution $\hat{y}$ to be close to the true distribution y .

1.3 Adam Optimizer

Adam (Adaptive Moment Estimation) is an optimization algorithm that adjusts the learning rate for each parameter using estimates of first and second moments of the gradients. The parameter update rule at each time step t is:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \\ \theta_t &= \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \end{aligned}$$

where g_t is the gradient at time t , m_t and v_t are the moving averages of the gradients and squared gradients, β_1 and β_2 are hyperparameters, and η is the learning rate. ϵ is a small constant to avoid division by zero.

1.4 Dropouts and Batch Normalization

Dropout randomly deactivates a subset of neurons during training to prevent overfitting. Mathematically, the output of a layer with dropout is given by:

$$\tilde{h}_i = h_i \cdot z_i \quad \text{where} \quad z_i \sim \text{Bernoulli}(p)$$

Here, h_i is the original activation, z_i is a binary mask sampled from a Bernoulli distribution with retention probability p . At test time, dropout is turned off and the activations are scaled accordingly.

Batch Normalization normalizes the activations of a layer across the mini-batch to have zero mean and unit variance, then scales and shifts them using learned parameters:

$$\text{BN}(x_i) = \gamma \cdot \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta$$

where x_i is the activation, μ_B and σ_B^2 are the mini-batch mean and variance, and γ, β are learnable parameters. ϵ is a small constant for numerical stability.

2. Dataset Details: FashionMNIST

Length of train data is 48000. Length of test data is 10000. Length of validation data is 12000.

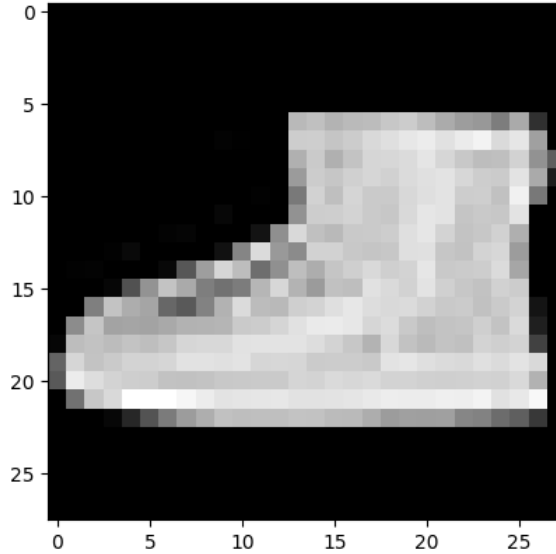


Fig. 1. Sample from FashionMNIST Dataset

3. Observations and Results

3.1 Model-1: Vanilla Neural Network without Dropout or Batch Normalization

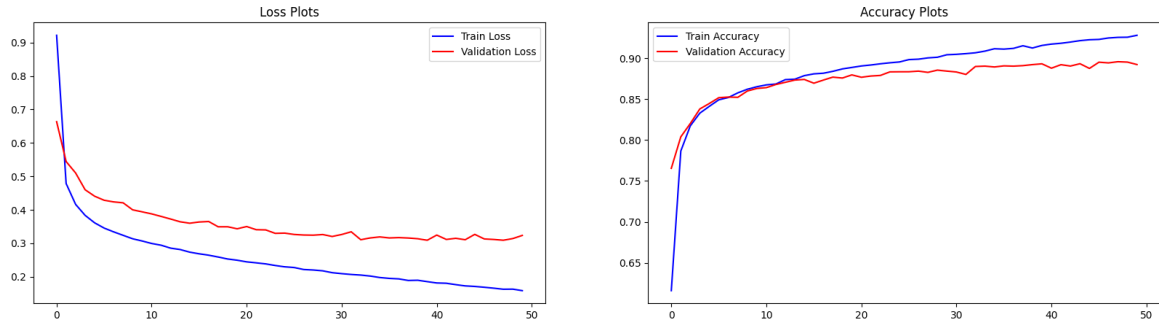


Fig. 2. Loss and accuracy plots for model-1

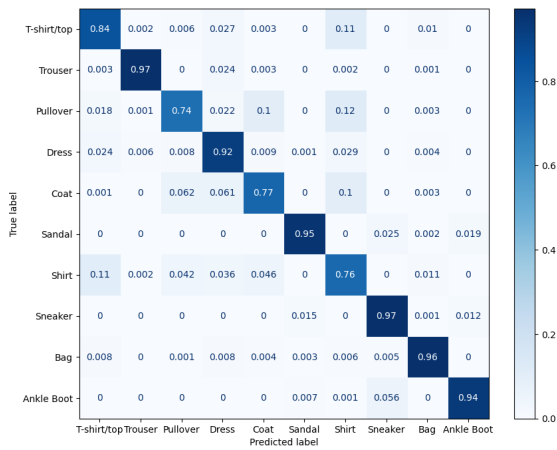


Fig. 3. Confusion matrix for model-1

Classification Report:				
	precision	recall	f1-score	support
0	0.84	0.84	0.84	1000
1	0.99	0.97	0.98	1000
2	0.86	0.74	0.80	1000
3	0.84	0.92	0.88	1000
4	0.82	0.77	0.80	1000
5	0.97	0.95	0.96	1000
6	0.67	0.76	0.71	1000
7	0.92	0.97	0.94	1000
8	0.96	0.96	0.96	1000
9	0.97	0.94	0.95	1000
accuracy			0.88	10000
macro avg	0.89	0.88	0.88	10000
weighted avg	0.89	0.88	0.88	10000

Fig. 4. Classification Report for model-1

3.2 Model-2: Vanilla Neural Network with Dropouts

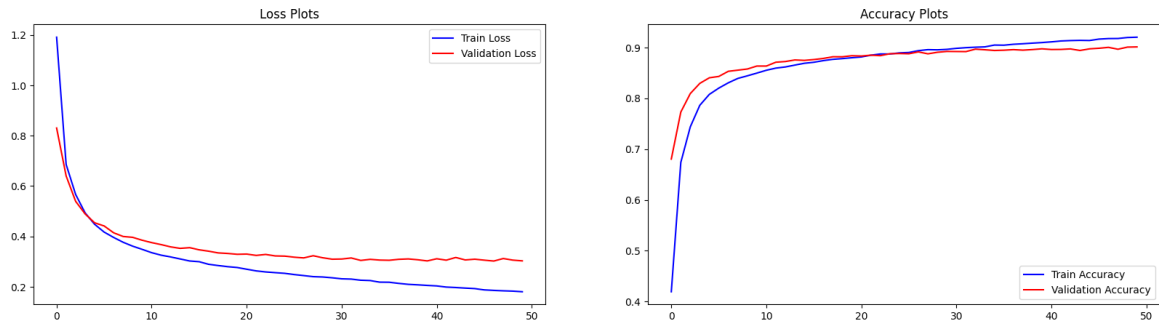


Fig. 5. Loss and accuracy plots for model-2

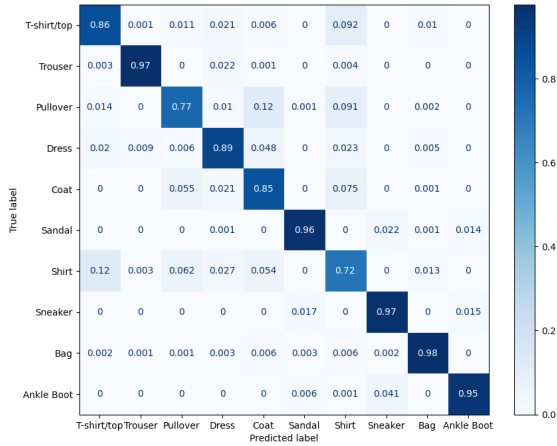


Fig. 6. Confusion matrix for model-2

Classification Report:

	precision	recall	f1-score	support
0	0.84	0.86	0.85	1000
1	0.99	0.97	0.98	1000
2	0.85	0.77	0.81	1000
3	0.89	0.89	0.89	1000
4	0.79	0.85	0.82	1000
5	0.97	0.96	0.97	1000
6	0.71	0.72	0.72	1000
7	0.94	0.97	0.95	1000
8	0.97	0.98	0.97	1000
9	0.97	0.95	0.96	1000
accuracy			0.89	10000
macro avg	0.89	0.89	0.89	10000
weighted avg	0.89	0.89	0.89	10000

Fig. 7. Classification Report for model-2

3.3 Model-3: Vanilla Neural Network with Batch Normalization

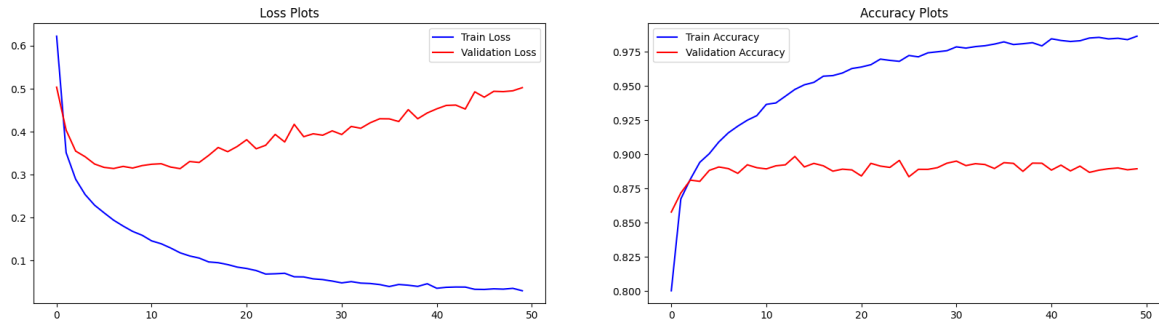


Fig. 8. Loss and accuracy plots for model-3

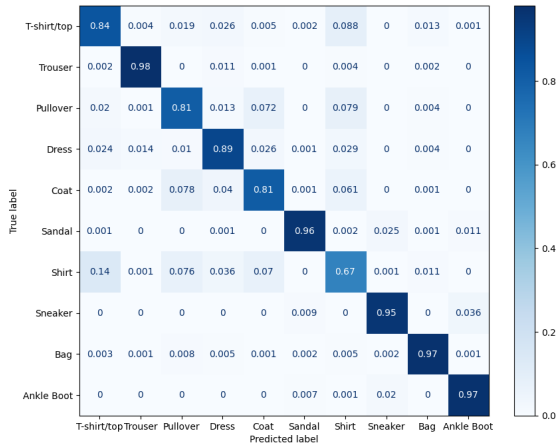


Fig. 9. Confusion matrix for model-3

Classification Report:

	precision	recall	f1-score	support
0	0.82	0.84	0.83	1000
1	0.98	0.98	0.98	1000
2	0.81	0.81	0.81	1000
3	0.87	0.89	0.88	1000
4	0.82	0.81	0.82	1000
5	0.98	0.96	0.97	1000
6	0.71	0.67	0.69	1000
7	0.95	0.95	0.95	1000
8	0.96	0.97	0.97	1000
9	0.95	0.97	0.96	1000
accuracy			0.89	10000
macro avg	0.89	0.89	0.89	10000
weighted avg	0.89	0.89	0.89	10000

Fig. 10. Classification Report for model-3

4. Discussion

4.1 Observation Discussion

From the experimental results, it is observed that Model-1 (without dropout or batch normalization) performs well initially but shows signs of overfitting, as evidenced by the widening gap between training and validation curves. Model-2, which incorporates dropout, demonstrates improved generalization and a narrower gap between training and validation accuracy, though its peak performance is slightly lower. Model-3, using batch normalization, achieves the best overall accuracy with smooth and stable convergence. It shows both high training and validation accuracy but with high overfitting, indicating that batch normalization is more effective than dropout in this setup to improve model performance, but dropouts prove to be more effective to handle overfitting.

4.2 Further Improvements

To further improve performance, Convolutional Neural Networks (CNNs) can be explored. Unlike fully connected layers, CNNs preserve spatial hierarchies and local features in the input image, making them well-suited for image classification tasks. By leveraging convolutional filters, pooling, and spatial locality, CNNs can extract richer features, reduce parameter count, and improve generalization, especially on structured image data like FashionMNIST.